

ARM PrimeCell

Advanced Audio CODEC Interface (PL041)

Integration Manual

ARM PrimeCell Advanced Audio CODEC Interface (PL041) Integration Manual

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Advanced Audio CODEC Interface

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	3
4.1	Design Flow Supported	3
4.2	Signal Connectivity	4
4.3	Bus Interconnection	6
4.4	Customisation	6
4.4.1	DMA Support	6
4.4.2	FIFO Sizes	7
4.4.3	FIFO Fill levels interrupts	7
4.5	Synthesis	7
4.5.1	Setup Scripts	7
4.5.2	Command Script	8
4.5.3	Constraint Scripts	8
4.6	Integration with AMBA Micropack components	9
4.7	Interrupt Connection	9
4.8	DMA Interface	9
4.9	CODEC Connection	9
4.10	Clocking	10
4.10.1	Bus Clock (PCLK) and AC-Link Clock (AACIBITCLK)	10
4.10.2	Inverted AC-Link Clock (nAACIBITCLK)	11
4.11	Clock Tree synthesis	11
4.12	Resets	11
4.13	Reset Buffering	13
4.14	Synchronisers	13

4.15	Endianness	13
4.16	Integration Testing	13
4.17	Production Testing	14
4.17.1	Scan Insertion and ATPG	14
4.18	Functional and Timing Verification	14
5	APPENDIX A	15
5.1	PrimeCell Environment Variables	15

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A00	01 June, 2000	First draft
A01	05 June, 2000	Review comments incorporated
A	05 September, 2000	First Release

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0173	ARM PrimeCell Advanced Audio CODEC Interface (PL041) Technical Reference Manual
3	PL041 DDES 0000	ARM PrimeCell Advanced Audio CODEC Interface (PL041) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	ARM DAI 0005	AMBA Interconnection schemes Application Note 05
7	SCB00-QPRO-0008-A03	ARM PrimeCell Integration Vector Strategy

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
AACI	Advanced Audio CODEC Interface

2 SCOPE

This document describes how the ARM PrimeCell Advanced Audio CODEC Interface (AACI PL041) may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell AACI is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell block, refer to the ARM PrimeCell Advanced Audio CODEC Interface (PL041) Technical Reference Manual [Ref.2].

The ARM PrimeCell Advanced Audio CODEC Interface (PL041) Design Manual [Ref.3] contains information about the implementation and design of the ARM PrimeCell AACI block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

The AMBA Interconnection schemes Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tri-state within an AMBA system [Ref.6].

The PrimeCell Integration Vector Strategy document [Ref. 7] describes the motivation and use of integration test vectors.

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell AACI has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell AACI block.

An example ASIC design flow is described below:

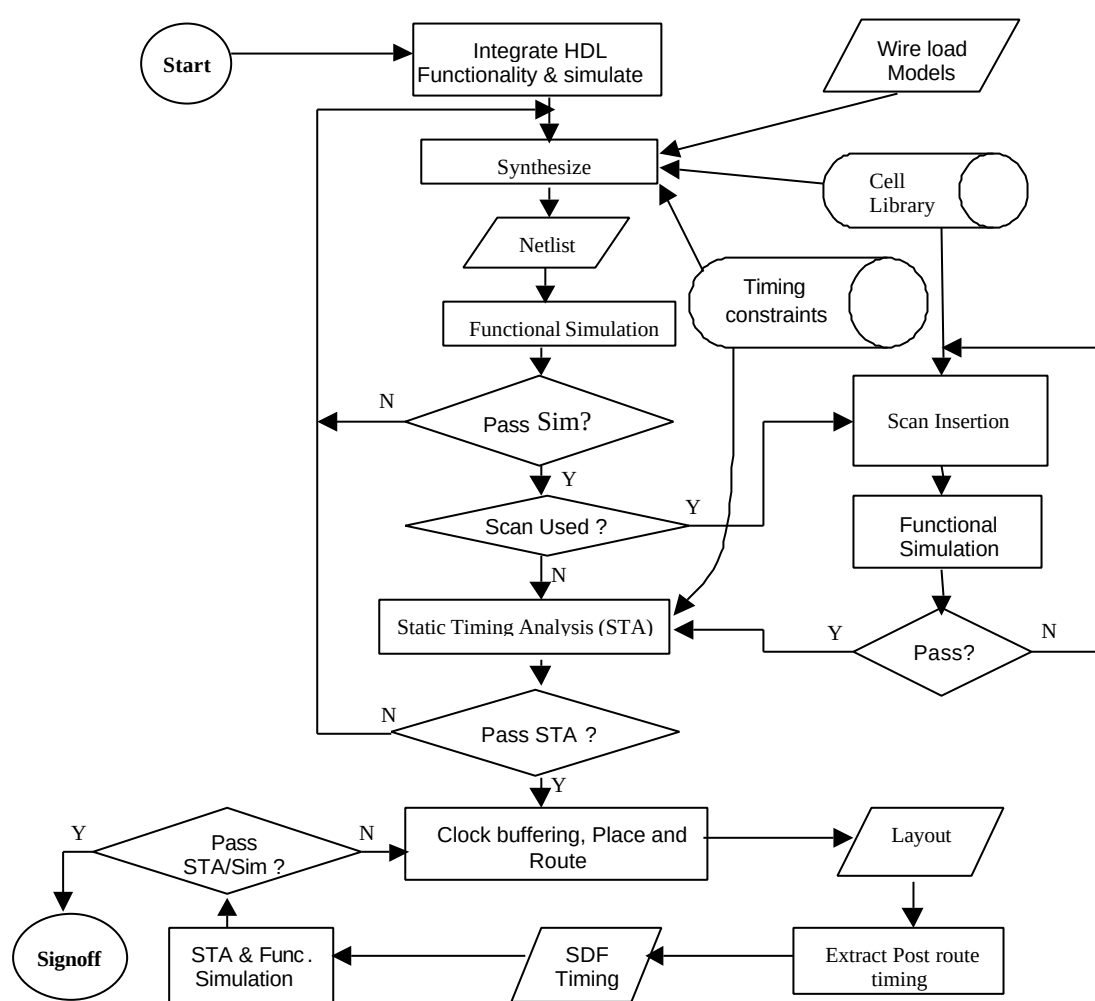


Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell AACI block. For further information refer to the ARM PrimeCell Advanced Audio CODEC Interface (PL041) Technical Reference Manual [Ref.2].

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB Clock.
PRESETn	IN	Reset Controller	APB Reset (active low).
PWRITE	IN	APB Bridge	APB write signal
PENABLE	IN	APB Bridge	APB enable
PSEL	IN	APB Bridge	APB peripheral select
PADDR[11:2]	IN	APB Bridge	APB address
PWDATA[31:0]	IN	APB Bridge	APB write data
PRDATA[31:0]	OUT	APB Bridge	APB read data

Table 4.1-1 APB interface signals

Signal Name	Type	Source / Destination	Description
nAACIBITCLK	IN	Clock Generator	Inverted AACIBITCLK.
nAACIBITCLKRST	IN	Reset Controller	Reset signal to AACIBITCLK Clock domain, active LOW.
nFAACIBITCLKRST	IN	Reset Controller	Reset signal to nAACIBITCLK Clock domain, active LOW.
SCANENABLE	IN	Test Controller	Scan Enable signal for all clock domains.
SCANINPCLK	IN	PCLK scan chain	Scan data Input signal for PCLK domain.
SCANINBITCLK	IN	AACIBITCLK scan chain	Scan data Input signal for AACIBITCLK domain.
SCANINnBITCLK	IN	nAACIBITCLK scan chain	Scan data Input signal for nAACIBITCLK domain.
SCANOUTPCLK	OUT	PCLK scan chain	Scan data Output signal for PCLK domain.
SCANOUTBITCLK	OUT	AACIBITCLK scan chain	Scan data Output signal for AACIBITCLK domain.
SCANOUTnBITCLK	OUT	nAACIBITCLK scan chain	Scan data Output signal for nAACIBITCLK domain.
AACIRXINTR1	OUT	Interrupt Controller	Receive FIFO 1 Interrupt.
AACIRXINTR2	OUT	Interrupt Controller	Receive FIFO 2 Interrupt.
AACIRXINTR3	OUT	Interrupt Controller	Receive FIFO 3 Interrupt.
AACIRXINTR4	OUT	Interrupt Controller	Receive FIFO 4 Interrupt.
AACITXINTR1	OUT	Interrupt Controller	Transmit FIFO 1 Interrupt.
AACITXINTR2	OUT	Interrupt Controller	Transmit FIFO 2 Interrupt.
AACITXINTR3	OUT	Interrupt Controller	Transmit FIFO 3 Interrupt.
AACITXINTR4	OUT	Interrupt Controller	Transmit FIFO 4 Interrupt.
AACIRXTOINTR1	OUT	Interrupt Controller	Channel 1 Receive Timeout Interrupt.
AACIRXTOINTR2	OUT	Interrupt Controller	Channel 2 Receive Timeout Interrupt.

AACIRXTOINTR3	OUT	Interrupt Controller	Channel 3 Receive Timeout Interrupt.
AACIRXTOINTR4	OUT	Interrupt Controller	Channel 4 Receive Timeout Interrupt.
AACIRXTOFEINTR1	OUT	Interrupt Controller	Channel 1 Receive Timeout FIFO Empty Interrupt.
AACIRXTOFEINTR2	OUT	Interrupt Controller	Channel 2 Receive Timeout FIFO Empty Interrupt.
AACIRXTOFEINTR3	OUT	Interrupt Controller	Channel 3 Receive Timeout FIFO Empty Interrupt.
AACIRXTOFEINTR4	OUT	Interrupt Controller	Channel 4 Receive Timeout FIFO Empty Interrupt.
AACITXCINTR1	OUT	Interrupt Controller	Channel 1 Transmit Complete Interrupt.
AACITXCINTR2	OUT	Interrupt Controller	Channel 2 Transmit Complete Interrupt.
AACITXCINTR3	OUT	Interrupt Controller	Channel 3 Transmit Complete Interrupt.
AACITXCINTR4	OUT	Interrupt Controller	Channel 4 Transmit Complete Interrupt.
AACIURINTR1	OUT	Interrupt Controller	Transmit FIFO 1 Underrun Interrupt.
AACIURINTR2	OUT	Interrupt Controller	Transmit FIFO 2 Underrun Interrupt.
AACIURINTR3	OUT	Interrupt Controller	Transmit FIFO 3 Underrun Interrupt.
AACIURINTR4	OUT	Interrupt Controller	Transmit FIFO 4 Underrun Interrupt.
AACIORINTR1	OUT	Interrupt Controller	Receive FIFO 1 Overrun Interrupt.
AACIORINTR2	OUT	Interrupt Controller	Receive FIFO 2 Overrun Interrupt.
AACIORINTR3	OUT	Interrupt Controller	Receive FIFO 3 Overrun Interrupt.
AACIORINTR4	OUT	Interrupt Controller	Receive FIFO 4 Overrun Interrupt.
AACIWINTR	OUT	Interrupt Controller	Wakeup Interrupt.
AACIGPIOINTR	OUT	Interrupt Controller	GPIO Interrupt.
AACIS1RXINTR	OUT	Interrupt Controller	Slot 1 Rx Interrupt.
AACIS1TXINTR	OUT	Interrupt Controller	Slot 1 Tx Interrupt.
AACIS2RXINTR	OUT	Interrupt Controller	Slot 2 Rx Interrupt.
AACIS2TXINTR	OUT	Interrupt Controller	Slot 2 Tx Interrupt.
AACIS12RXINTR	OUT	Interrupt Controller	Slot 12 Rx Interrupt.
AACIS12TXINTR	OUT	Interrupt Controller	Slot 12 Tx Interrupt.
AACIINTR	OUT	Interrupt Controller	Combined AACI interrupt.
AACIDMASREQRX	OUT	DMA Controller	Single Word DMA receive transfer request.
AACIDMALSREQRX	OUT	DMA Controller	Last Single Word DMA receive transfer request.
AACIDMABREQRX	OUT	DMA Controller	Burst DMA receive transfer request.
AACIDMALBREQRX	OUT	DMA Controller	Burst DMA receive transfer request.
AACIDMACLRXX	IN	DMA Controller	DMA Receive Request Clear.
AACIDMABREQTX	OUT	DMA Controller	Burst DMA transmit transfer request.
AACIDMACLRXX	IN	DMA Controller	DMA Transmit Request Clear.

Table 4.2-2: On-Chip Signals

Signal Name	Type	Source / Destination	Description
AACIBITCLK	IN	PAD	Clock supplied by the Audio CODEC, fixed at 12.288 MHz.
AACISYNC	OUT	PAD	AACIBITCLK divided by 256. (48 kHz). Also output asynchronously by the AACI when Audio CODEC is in Warm reset state.
AACIRESET	OUT	PAD	Reset signal from the AACI to the Audio CODEC.
AACISDATAIN	IN	PAD	Serial Data In from the Audio CODEC.
AACISDATAOUT	OUT	PAD	Serial Data Out to the Audio CODEC.

Table 4.2-3: Signals to I/O pad cells

4.3 Bus Interconnection

The AACI block interfaces with the AMBA APB interface using separate uni-directional read (output) and write (input) data buses PRDATA and PWDATA. For further information refer to the AMBA APB Specification [Ref.1].

The data bus may be interfaced at the APB Bridge either using multiplexers or logical OR'ing of data buses from other APB peripheral blocks. In common with other APB peripheral blocks, the AACI block forces all read data bus lines low when there are no APB read transfers in progress.

However, it is also possible to interface the AACI block with a 3-state bi-directional data bus. A wrapper needs to be added around the AACI block connecting the read and write data buses to bi-directional drivers. Control of the 3-state drivers can be done using the APB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and achieving high coverage fault grading.

4.4 Customisation

The ARM PrimeCell AACI can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Information to support such modification is included in the ARM PrimeCell Advanced Audio CODEC Interface (PL041) Design Manual [Ref.3].

4.4.1 DMA Support

The AACI implements four channels – allowing simultaneous support for four different sample rates. All channels can be serviced by the processor. Channel 1 however can also support DMA accesses. Hence the DMA Receive Channel (AaciDMARChannel) and DMA Transmit Channel (AaciDMATxChannel) are instantiated only once for channel 1 in the top level module/entity (Aaci) of the HDL source code while the non-DMA Recieve Channel (AaciRxChannel) and non-DMA Transmit Channel (AaciTxChannel) are instantiated three times for channels 2, 3 and 4. If more than one channel is required to support DMA accesses, it can be done by instantiating the DMA Receive Channel (AaciDMARChannel) and DMA Transmit Channel (AaciDMATxChannel) more than once in the top level module/entity (Aaci). If this is done, the additional DMA channels will require a set of DMA request and clear pins. Additional register bits will also have to be added if each Channel is to be enabled independent of the others. The testbench, trickbox and the functional test program must also be modified accordingly.

4.4.2 FIFO Sizes

The Tx and Rx FIFO sizes of all the channels are defined in the HDL source code to be 20-bit wide and 8 locations deep. These sizes should provide reasonable interrupt latency, adequate for most systems. If different size FIFO buffers are required, for example, to increase allowable latency before overflow or underflow occurs, the FIFO register file and control logic blocks (AaciTxFCntI, AaciDMATxFCntI, AaciRxFCntI, AaciDMARxFCntI, AaciTxRegFile and AaciRxRegFile) can be modified accordingly. If this is done, the functional test program must also be modified.

4.4.3 FIFO Fill levels interrupts

The interrupt logic generates two interrupt request signals from each channel depending on the FIFO Fill levels – AACIRXINTR and AACITXINTR. The FIFO interrupt generation thresholds are set at the half way levels for both the Receive (Rx FIFO Half Full) and Transmit (Tx FIFO Half Empty) cases. This allows the best compromise between good interrupt latency and efficient burst transfer of data across the bus. The fill levels can be modified in the FIFO control logic blocks (AaciRxFCntI, AaciDMARxFCntI, AaciTxFCntI, AaciDMATxFCntI). If this is done, the functional test program must be modified accordingly.

4.5 Synthesis

The AACI block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script - Aaci.cmd - synthesizes the AACI to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the Aaci_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

The synthesis scripts consist of the following three main categories:

4.5.1 Setup Scripts

Scripts that are common to several designs are located in the \$GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The global.scr script may need modifying to define:

- target area goals
- maximum transition costs

The library_avanti_cb25_v2.1.scr script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the library_avanti_cb25_v2.1.scr file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The \$LIB_SYNOP directory must contain the files, or links to the target cell library files, in Synopsys format.

4.5.2 Command Script

The master compile command script `Aaci.cmd` in the `synopsys/scripts` directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The `Aaci.cmd` script includes setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys `.db` format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate `log/` directory.

The scan-insertion script `Aaci_scan.cmd` in the `synopsys/scripts` directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- `$PERIPH` is assigned to 'topname' to define the name of the top level module in the design.
- `$HDL_SOURCE` is assigned to 'hdl' to define the input HDL source. `HDL_SOURCE` should be set to either `vhdl` or `verilog` in order to read in the VHDL or Verilog source RTL files.
- `$HDL_NETL` is assigned to 'hdl_out' to define the output format for netlist and SDF files. It can be set to either `vhdl` or `verilog` to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

4.5.3 Constraint Scripts

- a) Operating conditions and signal load/drive values should be set for the chosen technology library in a file equivalent to the `library_avanti_cb25_v2.1.scr` file.
- b) Timing constraints for APB bus signals are set via the `apb_params.scr` and the `apb_slave.scr` file. As delivered, the timings are set for a 100 MHz clock. These timings should be adjusted according to the target frequency required. The `apb_params.scr` file has been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- the minus uncertainty value is set at 5% of the clock period.
- the plus uncertainty value is set at 40% of the minus uncertainty value.

The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

The timing parameters assume a bus timing budget of 20% for output delays and 40% for input setup. This is a reasonable estimate assuming that the process technology is such that APB Bridge can meet this constraint with signals being available after 40% of the clock period.

If a very high frequency bus clock is being used, it is important to ensure that the read data output timing is adequate to meet the requirements of the APB Bridge. In some designs the read data may not be sampled by the APB Bridge, but instead passed directly to the system bus. In this case, it is necessary to ensure that the timing requirements are met for the block which is sampling the data.

The value of 40% of clock period allowed for input setup of APB inputs applies for a style of APB Bridge design with registered inputs implementing logic after the registers. If the APB Bridge has registered outputs then the input setup can be increased, which should result in a reduction of area required.

- c) Timing constraints for non-AMBA AACI-specific signals are set via the `Aaci.scr` file. The `AACIBITCLK` frequency is set to 12.5 MHz. This is marginally higher than the AC-Link specified frequency of 12.288 Mhz. Sample input and output delay constraints are also specified for the other signals. It should be noted that for asynchronous signals, the timing budgets are arbitrary since they can never be accurate. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.
- d) Timing exceptions are specified via the `Aaci_exceptions.scr` file. False paths are specified between the APB clock domain and the AC-Link clock domain to account for signals passing through synchronisers as well as unsynchronised paths created due to the Integration Test strategy.

4.6 Integration with AMBA Micropack components

The AACI block is designed to be integrated into an AMBA system via the APB (Advanced Peripheral Bus.) It is designed to work in systems where an AMBA bus master can read from or write to the AACI through the APB bridge (from the ASB or AHB) in response to interrupt signals. A dedicated DMA interface is also available to enable connection to external DMA controllers.

An example APB Bridge is available within the ARM Micropack product. The decode logic within the APB Bridge will need to be altered to take into account the address location of the AACI and any other APB peripherals in the memory map. The base address of the AACI needs to be defined on a system specific basis and the bridge decode logic altered accordingly. The APB Bridge appears as a slave peripheral on the ASB / AHB with a decode signal, which must be altered in the HDL code of the AMBA decoder block, to select the correct range for the APB peripherals (including the AACI). If AMBA TIC methodology is to be used for integration testing then any change of address location for the AACI block must also be reflected in the AMBA TIC test vectors, by amending the base address declaration for the AACI integration test program.

4.7 Interrupt Connection

The PrimeCell AACI generates individual maskable active HIGH interrupts. Each interrupt can be enabled or disabled by changing its enable bit in the mask register. Setting the enable bit HIGH enables the corresponding interrupt. This allows the system interrupt controller to provide the mask registers for each interrupt. The interrupts are also output as a combined single interrupt, AACIINTR that is the OR function of the individual masked interrupts. This output can be connected to the system interrupt controller to provide another level of masking on a per-peripheral basis. The status of the individual interrupt sources can be read from the appropriate register.

The AACIINTR would typically be routed to the nIRQ interrupt line of the ARM processor through an external Interrupt Controller.

4.8 DMA Interface

Channel 1 of the AACI provides a dedicated DMA interface to connect to PrimeCell DMA controllers. [Please refer to the ARM PrimeCell Advanced Audio CODEC Interface (PL041) Technical Reference Manual [Ref.2] for a description of the DMA interface.]

If the AACI is being integrated into a system where the dedicated DMA interface is not used, the DMAEnable bit in the AACIMAINCR register should be cleared.

4.9 CODEC Connection

The PrimeCell AACI supports connection with up to 4 Audio CODECs on an extended AC-Link of which there is at least one Primary CODEC and up to 3 Secondary CODECs. Saving of AACI pins is attained by connecting a single set of AC-Link signals (AACIBITCLK, AACISDATAOUT, AACISDATAIN, AACISYNC and AACIRESET) to all the 4 CODECs. All the Secondary CODECs' BITCLK input and the AACIBITCLK input of the AACI are connected to the BITCLK output from the Primary CODEC. The SDATAIN outputs of the four CODECs must be connected to the single input AACISDATAIN of the AACI through an OR gate. The connection of the AC-Link signals to the Primary and Secondary CODECs is shown in **Figure 4.9-1**.

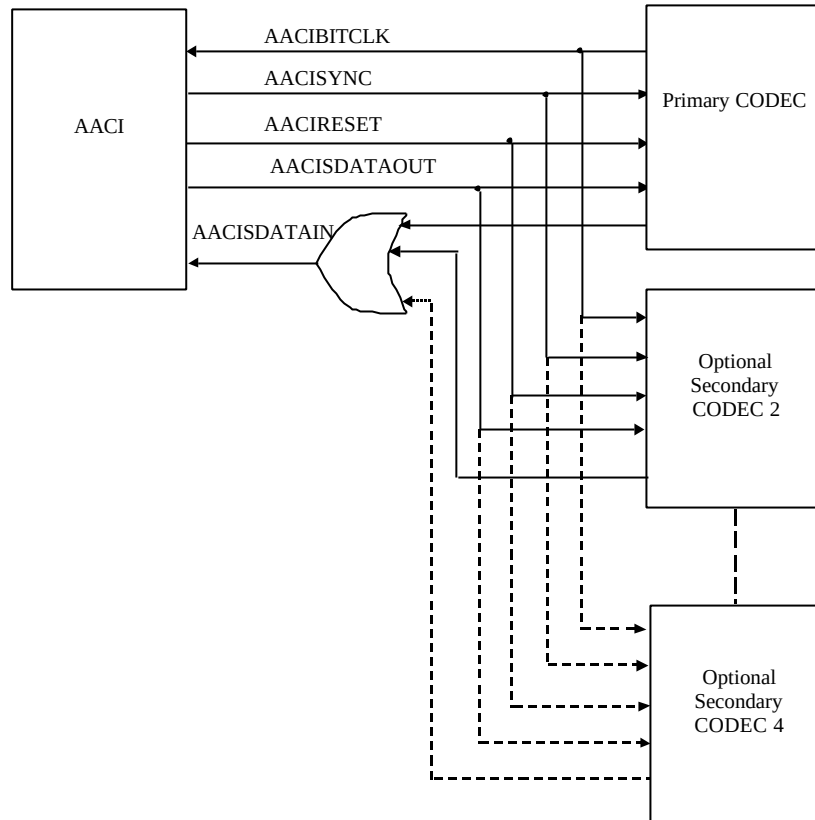


Figure 4.9-1 AACI connection with Primary CODEC and Secondary CODECs

4.10 Clocking

4.10.1 Bus Clock (PCLK) and AC-Link Clock (AACIBITCLK)

The ARM PrimeCell AACI HDL contains two primary clock sources at the top level – PCLK and AACIBITCLK. AACIBITCLK is the AC-Link Clock supplied by the Audio CODEC. This clock is fixed at 12.288MHz.

Since separate asynchronous clocks are used, each clock domain can be individually statically analysed and the simplest strategy is to use two separate scan chains with no mixing of clocks or edges. It is also possible to implement a single scan chain mixing different clocks and scan tool vendor documentation should be consulted for further details.

The following PCLK to AACIBITCLK frequency ratios have to be satisfied when both AACIBITCLK and PCLK are free running.

PCLK faster than AACIBITCLK

$$f(\text{PCLK}) > f(\text{AACIBITCLK})$$

PCLK slower than AACIBITCLK

$$f(\text{PCLK}) \geq f(\text{AACIBITCLK} / 5)$$

It should be noted that the AACI design assumes that both the clock sources are free-running during normal device operation. While gating the clocks is not recommended, it might have to be employed in very power critical

applications. It is imperative that the PCLK is shut off only after the AACI is completely idle. Accesses to the AACI are not guaranteed to complete correctly if PCLK is not free-running.

4.10.2 Inverted AC-Link Clock (nAACIBITCLK)

For functional reasons, the AACI logic requires to use the falling edge of nAACIBITCLK. Since PrimeCell design rules prohibit the use of negative edge triggered F/Fs, the inverted version of AACIBITCLK is brought in as an input to the AACI. The creation of the inverted AC-Link clock has to be done external to the AACI block during integration.

4.11 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default AACI synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools which understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.12 Resets

The AACI block has one reset pin for each of its internal clock domains. The system bus reset signal PRESETn, resets the PCLK domain registers. A separate reset pin nAACIBITCLKRST, is used to reset the AACIBITCLK domain registers and nFAACIBITCLKRST is used to reset the nAACIBITCLK domain registers. The nAACIBITCLKRST and nFAACIBITCLKRST signals need to be provided by a separate Reset Controller, and they must be de-asserted synchronously to AACIBITCLK and nAACIBITCLK respectively. The resets can be asserted asynchronously even if no clocks are present to ensure that signals from bi-directional I/O cells have defined values during reset, thus avoiding static power consumption problems. While re-synchronisation of PRESETn to AACIBITCLK could have been done within the AACI block itself, the function was moved out to eliminate internally generated resets.

Figure 4-12-1 illustrates a method of generating nAACIBITCLKRST from PRESETn, such that it will be asserted asynchronously and de-asserted synchronous to AACIBITCLK. The reset signal nAACIBITCLKRST is de-asserted two AACIBITCLK cycles after PRESETn is de-asserted to prevent meta-stability problems

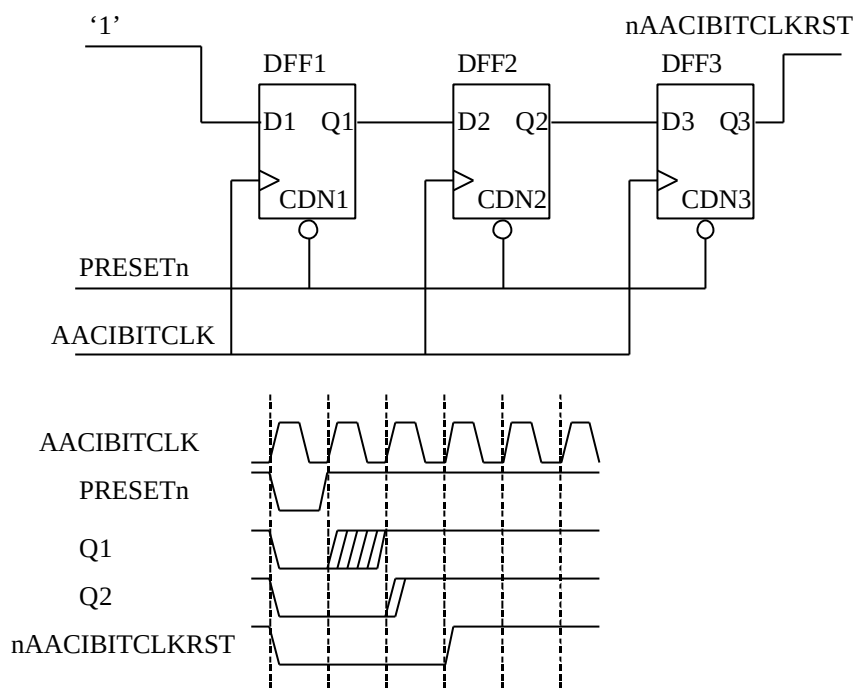


Figure 4.12-1 A method of generating $nAACIBITCLKRST$

The method of generating $nAACIBITCLKRST$ in **Figure 4.12-1** could experience simulation problems depending on the cell library modelling. The Q output of DFF3 should not be meta-stable when setup/hold violation occurs due to de-assertion of $PRESETn$, since both D and Q are both '0', however the DFF model may assign 'X'.

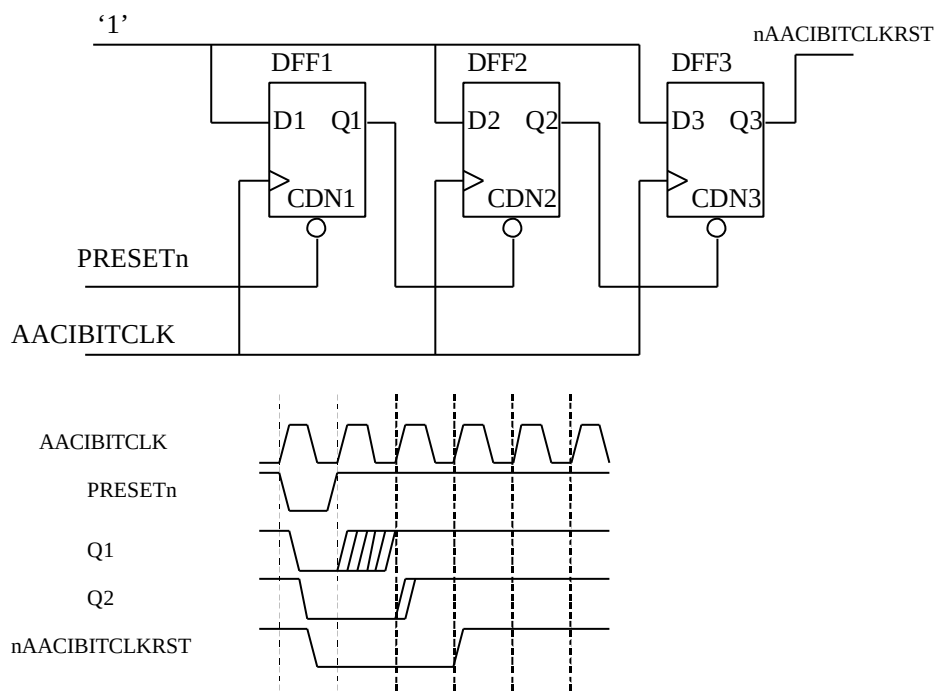


Figure 4.12-2 An alternative method of generating $nAACIBITCLKRST$

An alternative method of generating nAACIBITCLKRST is therefore illustrated in **Figure 4.11-2**, which achieves the same functionality. This method does use internally generated resets, which can present problems in Static Timing Analysis, scan insertion and ATPG, but they are limited to the reset controller at the top-level of the design hierarchy.

nFAACIBITCLKRST is also generated using the method used for generating nAACIBITCLKRST such that it is asserted asynchronously and deasserted synchronous to nAACIBITCLK.

4.13 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

4.14 Synchronisers

Single bit signals between the two clock domains are synchronised by two series flip-flops inferred during synthesis from standard HDL templates. Multiple bit vector signals use an update mechanism with synchronised single bit control signals to avoid meta-stability problems.

Some technology libraries offer meta-stability hardened D flip-flops specifically for implementing synchronisers. All the synchronisers in the AACI block have been isolated to the sub-blocks AciPtoBSync and AciBtoPSync. This allows the synthesis scripts to be modified so that these blocks could be synthesized separately to use the synchroniser flip-flop cells using the Synopsys Design Compiler `set_prefer` command. Further synthesis of these sub-blocks can then be prevented using the `set_dont_touch` command. The rest of the AACI design is then synthesized after placing a `set_dont_use` command on the synchroniser cells. This method also offers the opportunity to simulate the gate-level netlist with asynchronous clocks PCLK and AACIBITCLK and eliminate the expected setup and hold violations on the synchronisers from the simulation transcript, since the synchronisers cells can be modified to remove 'X' propagation.

4.15 Endianness

The AACI supports only 32-bit accesses on its APB data buses. For a 32-bit system, the AACI is bi-endian to the extent that it can support 32-bit accesses from a little-endian master and a big-endian master. In systems with wider data buses, data routing will have to be done external to the AACI.

4.16 Integration Testing

The AACI integration test vector suite allows the integrator to verify that the AACI has been connected into the target system correctly. These vectors test 3 classes of signals:

- a) AMBA signals (AACI signals connected directly to the APB)
- b) Intra-Chip signals (AACI signals connected to the DMA and Interrupt Controllers)
- c) Primary I/O signals (AACI signals connected to the chip pads)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 7].

The integration vectors are contained in the `IntegrationTest.c` file in the `integration/bustest` directory. The `AddrRangeTest.c` and `RegisterTest.c` files in the same directory contain the vectors which are used to increase the toggle coverage of the tests. The file `Aaci.c` in the `integration/bustest` directory is used to selectively run some

or all of these tests. As delivered, these vectors are intended to run correctly with the AACI in stand-alone mode i.e even before the AACI has been integrated into the system. After integration, the following points need to be noted:

- a) The integration vectors targeting the AACI to APB (AMBA signals) connectivity do **NOT** have to be modified by the integrator.
- b) The integration vectors targeting the AACI to DMA and AACI to Interrupt Controller (Intra-Chip signals) will **NEED** modification. The IntegrationTest.c file has been embedded with NOTES listing the changes that the integrator would have to make in the test code.
- c) The integration vectors targeting the AACI to Pad connectivity do **NOT** have to be modified by the integrator. However it is imperative that the integrator connect the Integration Trickbox correctly to these pads. [Please refer to the sample integration testbench – tbench.vhd[v] – in the integration/vhdl[verilog]/tbench directory]. It is also essential that during integration testing **ONLY** the Integration Trickbox is connected to these pads.

4.17 Production Testing

The AACI is designed to be production tested using scan-insertion and ATPG.

4.17.1 Scan Insertion and ATPG

It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The AACI design fully supports this scan test and ATPG methodology. Depending on the clocking arrangement used (see **section 4.9**), the number of chip level scan chains required may vary. As delivered the AACI is synthesised with 3 scan-chains – one each for the PCLK, AACIBITCLK and nAACIBITCLK domains. By having dedicated scan chains for each clock domain [rather than mix them internal to the AACI], complete freedom has been given to the integrator to decide chip level scan-chain ordering.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flips-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The main advantage of the scan insertion and ATPG approach is in cases where significant modifications have been made to the AACI block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that a very high fault coverage can be achieved (99%).

Extraction of test vectors is done by the system designer once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running the automatically generated vectors (ATPG).

4.18 Functional and Timing Verification

It is essential to verify timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC.

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results tend to be optimistic since worst case timing paths may not be exercised.

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis. Use of gate-level simulation as well as Static Timing Analysis is a useful check for human error in the manual definition of false paths.

The netlist should also be checked for functional equivalence to the RTL at the same key stages of the design flow.

This can be performed by dynamic simulation using test vectors as described above, or by use of a logic equivalence checking tool (For example, Synopsys Formality or Chrysalis).

5 APPENDIX A

5.1 PrimeCell Environment Variables

Simulation and Synthesis in the Aaci world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the '**GLOBAL**' variable.

eg : **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the '**PERIPH**' variable.

eg: **setenv PERIPH Aaci**

SIMULATOR

The simulator type is defined by the '**SIMULATOR**' variable.

It can take the following two values

- modelsim - for ModelSim simulations
- LDV - for LDV simulations

eg : **setenv SIMULATOR modelsim**

setenv SIMULATOR LDV

TEST_METH

The scan methodology adopted during synthesis is defined by the '**TEST_METH**' variable.

It can take the following three values

- noscan - creates a netlist without scan structures
- scanready - creates a netlist with scan F/Fs but without scan routing.

- scaninsert - creates a fully routed scan netlist.

eg : **setenv TEST_METH noscan**

setenv TEST_METH scanready

setenv TEST_METH scaninsert

HDL_SOURCE

The source code type is defined by the '**HDL_SOURCE**' variable.

It can take the following two values

- vhdl [VHDL RTL]

- verilog [Verilog RTL]

eg : **setenv HDL_SOURCE vhdl**

setenv HDL_SOURCE verilog

HDL_NETL

The netlist type is defined by the '**HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]

- verilog [Verilog netlist]

eg : **setenv HDL_NETL vhdl**

setenv HDL_NETL verilog

TEST_ENV

The test environment is defined by the '**TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]

- TICTALK [For integration vector simulations]

eg : **setenv TEST_ENV BUSTALK**

setenv TEST_ENV TICTALK

LIB_VHDL

This location of the vhdl cell library views is defined by the '**LIB_VHDL**' variable.

eg : **setenv LIB_VHDL /AVANT/1999.1/cb25/v2.1/vital/cb25os163/zero**

DIRS_VERILOG

This location of the verilog cell library views is defined by the '**DIRS_VERILOG**' variable.

eg : **setenv DIRS_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero**

FILES_VERILOG

This location of the verilog UDP file is defined by the '**FILES_VERILOG**' variable.

eg : **setenv FILES_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v**

LIB_SYNOP

This location of the synopsys target library files is defined by the '**LIB_SYNOP**' variable.

eg: **setenv LIB_SYNOP /AVANT/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models**